

assignment

April 10, 2025

As we know, an assignment problem is a special linear program. In this project, we solve an assignment problem using the `scipy.optimize` package in two ways – as a general linear program and as an assignment problem.

```
[1]: import numpy as np
      from scipy.optimize import linprog, linear_sum_assignment
      import time

[2]: n = 200      # n: number of jobs/workers
      cost = np.random.randint(0, n**2, (n, n))    # generate an n-by-n random matrix
      # with integer entries from 0 to n^2-1
```

1 Task 1: (35 Points)

Let $C = [c_{ij}]$ be the cost matrix of an assignment problem. The problem can be modeled as a linear program:

$$\min \sum_{i=1}^n \sum_{j=1}^n (c_{ij} x_{ij})$$

subject to:

$$\sum_{i=1}^n x_{ij} = 1 \text{ for } j = 1, \dots, n$$

$$\sum_{j=1}^n x_{ij} = 1 \text{ for } i = 1, \dots, n$$

$$x_{ij} \geq 0 \text{ for } i, j = 1, \dots, n$$

```
[3]: c = cost.reshape((n**2,1))    # reshape the cost matrix to a column vector
```

Define `A_eq` and `b_eq` in the following cell. You may use multiple cells if needed. Your code should work for a general `n`. Use the following documents and/or other online resources for help.

- **scipy.optimize.linprog**: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>
- **Numpy basics**: https://numpy.org/doc/stable/user/absolute_beginners.html

Constraints:

- Each worker does exactly one job: $\sum_{j=1}^n x_{ij} = 1$ for each worker i (total of n constraints).
- Each job is assigned to exactly one worker: $\sum_{i=1}^n x_{ij} = 1$ for each job j (total of n constraints).

We have $2n$ constraints in total, each constraint is a row in `A_eq`... should have dimensions $(2n, n**2)$.

- Worker constraint: sum over each row = 1.
- Job constraint: sum over each col = 1.

```
[4]: # A_eq
A_eq = np.zeros((2*n, n**2))

# worker constraint
for i in range(n):
    for j in range(n):
        A_eq[i, n*i+j] = 1

# job constraint
for j in range(n):
    for i in range(n):
        A_eq[n+j, n*i+j] = 1

# RHS
b_eq = np.ones(2*n)
```

```
[5]: start_time = time.time()    # start timing

res = linprog(c, None, None, A_eq, b_eq)

end_time = time.time()    # end timing
elapsed_LP = end_time - start_time
```

```
[6]: print('---- linprog() ----')
print(f"Minimum cost: {res.fun}.")
print(f"Completed in {elapsed_LP} s.")
```

```
---- linprog() ----
Minimum cost: 66540.0.
Completed in 0.20350384712219238 s.
```

2 Task 2: (15 points)

Instead of solving the assignment problem as a linear program, the function `scipy.optimize.linear_sum_assignment()` solves the assignment problem directly. Your code should solve the problem with the cost matrix `cost` and record the elapsed time. If coded correctly, the minimum costs returned by the two methods should be identical (up to some tiny numerical error).

Use the following cell to solve the assignment problem with the cost matrix “cost” and record the elapsed time. You may use multiple cells if needed.

```
[7]: start_time = time.time()
row_ind, col_ind = linear_sum_assignment(cost) # compute total cost
cur_time = time.time() - start_time
```

```
min_cost = cost[row_ind, col_ind].sum() # compute min cost

print('---- linear_sum_assignment() ----')
print(f'Minimum cost = {min_cost}.')
print(f'Completed in {cur_time} s.')
```

```
---- linear_sum_assignment() ----
Minimum cost = 66540.
Completed in 0.0010004043579101562 s.
```

3 Task 3: (10 points)

Run the file with different number of jobs n . (You can click on Restart & Run All under the Kernel tab.) Record the elapsed time for both methods. Fill in the following table.

n (# of jobs)	10	20	50	100	200
LP time	0.0035s	0.00367s	0.0095s	0.0302s	0.2277s
assignment time	0.0010s	0.00099s	0.0009s	0.0009s	0.0009s

Which method is faster?

- Our `linear_sum_assignment()` function significantly outperforms original method of solving as linear program.

4 Submission.

Your submission should include two parts.

- Part I: Fill in the blank cells in 'assignment.ipynb' and submit the file to Canvas.
- Part II: Put down your answer to Task 3 (and any other comments) in a pdf file and submit it to Gradescope.

[]: